

협동 1 프로젝트

두산로보틱스 M0609를 이용한 지상형 태양광 패널 구조물 조립 MES 연동 자동화

F/T 센서 기반 힘 제어 삽입으로 구현한
비전 없는 협동로봇 조립 자동화 시스템

Rokey 부트캠프 협동 프로젝트 1 발표자료



목차

01	프로젝트 개요	기획 배경 · 핵심 공정 · 차별화 포인트
02	팀 구성 및 역할	레이어 기준 담당 업무 분배
03	수행 절차 및 방법	요구사항 추적 체계 · 구현 순서 · 일정
04	수행 경과	시스템 아키텍처 · 구현 결과 · 진행 이슈
05	자체 평가 의견	완성도 · 잘한 점 · 개선점 · 성과

프로젝트 개요

옥외 태양광 설치 현장의 특성을 반영한 의도적 설계 선택

1

프로젝트 주제 및 선정 배경

옥외 태양광 설치에 조명·먼지 변화가 커 비전 인식에 불리함.
또한, 지붕 추락과 전기 감전 등 작업자로부터 위험 요소 제거함.
정형화된 반복 공정에 F/T 센서 기반 접촉 감증을 적용한 차별화 사례



태양광 공사 현장에서 떨어짐을 예방해야 하는 이유

..... 최근 5년간 태양광 공사 사고사망 현황

최근 5년간 태양광 공사 사고사망자 수는 25명이며,
2024년 한해에만 떨어짐 사고 사망자 8명 발생(승인일자 기준)



최근 사고사망자 수 증가 이유

온실가스 감축 방안의 일환으로 정부 보조금 지원사업 확대 및
중·소 규모 사업장의 태양광 설비 설치 수요 증가

고소작업으로 인한 떨어짐 사고 증가 →

..... 최근 5년간 태양광 공사 사고사망 현황 분석

태양광 설비 시공 중 사망사고는 취약 지붕(채광창 등) 파손에 의한
떨어짐 사고가 대부분이며, 축사 또는 공장 등의 중·소규모 공사장에서 주로 발생

고용노동부·한국산업안전보건공단 자료

프로젝트 개요

옥외 태양광 설치 현장의 특성을 반영한 의도적 설계 선택

2

프로젝트 내용

기둥 - 프레임 - 스냅핏 체결 - 패널
설치 4단계 조립을 F/T 센서로
검증하며 자동 수행, MES 연동
대시보드로 공정 관리

3

프로젝트 구조

목적 : 비전없는 F/T 기반 조립 자동화 구현

기간: 2주 / 사전기획 ~ 통합 테스트

DashBoard - Backend - Bridge - Robot 4계층 구현

- 의존성
- ROS2 Jazzy
- Python 3
- Flask, Flask-SQLAlchemy, SQLAlchemy,
python-dotenv, psycpg2, requests
- PostgreSQL 16

프로젝트 개요

옥외 태양광 설치 현장의 특성을 반영한 의도적 설계 선택

보고서 속성	세부정보
기준 연도	2025
예측 기간	2026-2035
연평균 성장률	15.3%
기준 연도 시장 규모(2025년)	14억 6천만 달러
예측 연도 시장 규모(2035년)	60억 6천만 달러
지역 범위	- 북미(미국 및 캐나다) - 아시아 태평양(일본, 중국, 인도, 인도네시아, 말레이시아, 호주, 한국, 기타 아시아 태평양 지역) - 유럽(영국, 독일, 프랑스, 이탈리아, 스페인, 러시아, 북유럽, 기타 유럽 지역) - 라틴 아메리카(멕시코, 아르헨티나, 브라질, 기타 라틴 아메리카 지역) - 중동 및 아프리카(이스라엘, GCC, 북아프리카, 남아프리카, 기타 중동 및 아프리카 지역)

솔루션/사례	비용 절감	시간/속도	인력	비고
AES Maximo	기존 대비 50%	설치시간 50% 단축	-	2025년 100MW 설치 목표, 3년 내 최대 5GW 파이프라인, 美 백로그 약 4GW
독일 기업 (PV Kraftwerker, Gehrlicher) 로봇 (IEEE Spectrum 보도)	14MW 발전소 기준 200만 → 90만 (약 55%)	최대 8배 단축	35명 → 3명	100만 달러 미만, 1년 이내 투자비 회수 가능



출처: AES Maximo

4

출처: ResearchNester

활용 방안 및 기대 효과

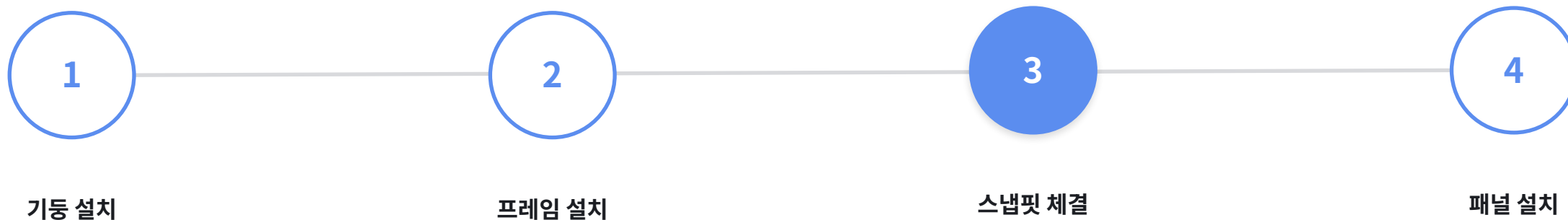
F/T 힘 제어 삽입 기술은 반사율 높은
옥외 부품 조립에 적용 가능하며,
협동로봇 자동화 사례로 활용 가능

Here's the executive summary, since I've never done an executive summary before and it sounds fancy: using robots to set up a 14 megawatt solar power plant can potentially cuts costs from \$2,000,000 to \$900,000, while being constructed in eight times faster with only three human workers instead of 35.

출처: IEEE Spectrum

핵심 공정 순서

기둥 설치부터 패널 고정까지, 4단계 순차 조립 프로세스



기둥과 주춧돌의 고정핀 체결은 로봇과의 협동 작업으로 사람이 작업하는 걸로 대체

제외 범위 비전 형상 인식 · 임의 위치 자동 탐색 · 자석 체결 · 볼트·너트 자동 체결

장비/환경 두산 M0609(그리퍼+내장 F/T) · Ubuntu 24.04 · ROS2 Jazzy · DART Platform/DRL · PostgreSQL

이 프로젝트의 핵심 기술

F/T 센서 기반 힘 제어 삽입

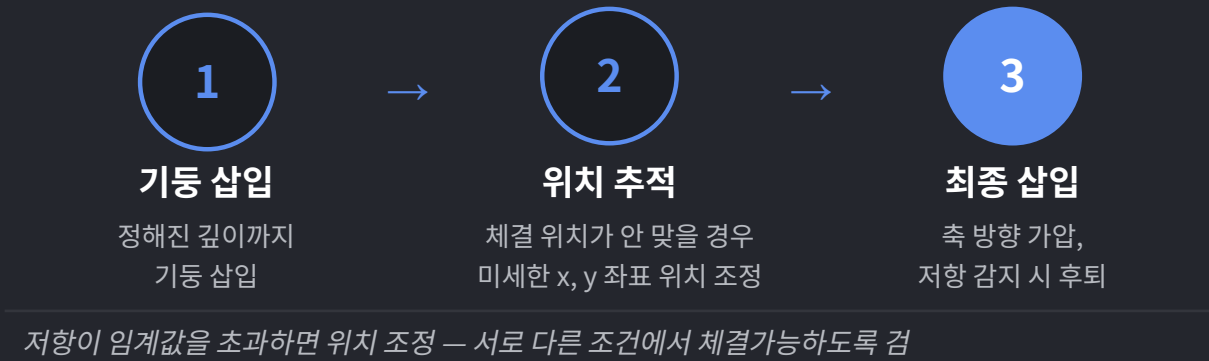
잠금핀을 정해진 깊이까지 1차 삽입한 뒤 재파지하여 축 방향으로 최종 삽입합니다. 과도한 저항이 감지되면 즉시 후퇴하여 안전을 확보합니다.

잠금핀을 정해진 깊이까지 1차 삽입한 뒤 재파지하여 축 방향으로 최종 삽입합니다. 과도한 저항이 감지되면 즉시 후퇴하여 안전을 확보합니다.

스냅핏 체결 프로세스



기둥 삽입 프로세스



팀 구성 및 역할

담당 범위는 저장소 4개 레이어와 매칭

백승민

팀장

Git, Notion 관리, 아키텍처 설계

봉승현

팀원

DB, Front·Backend 구현, ROS2-DB
연동

전제환

팀원

ROS2 및 DSR 움직임 구현, ROS2-DB
연동

이현민

팀원

Solid Works 하드웨어 설계

이일주

멘토

지원 내역 : 프로젝트 구현 조언

사전 기획 — 요구사항 추적 체계

BR → SYS-FR → FR → TC로 이어지는 추적 체계를 먼저 세운 뒤 구현 착수



4개 Actor 기반 인터페이스 설계

현장 관리자 · 협동로봇 작업 관리 노드 · DB · 협동로봇 제어 시스템 — 4개 Actor로 시스템을 나누고,
Actor 간 인터페이스(ROS2 Action/Service/Topic, DB API)를 먼저 문서로 확인한 뒤 구현에 착수

구현 순서 — 계약 우선 설계

통신 규격을 먼저 정의하고, 로봇 제어 → 중계 → 백엔드 순으로 구현

1

solar_panel_interface

Action/메시지/서비스 규격 정의 — 계약 우선 설계

2

solar_panel_robot

모션 → 힘 제어 → 공정 시나리오 → Action Server

3

ros2_bridge

HTTP 큐잉 → Action Goal 순차 전달 → 결과 콜백

4

web_app

Work Order/History 관리 + PostgreSQL(9테이블) + 대시보드

프로젝트 일정



총 개발기간 [총 2주]

시스템 아키텍처

Dashboard부터 로봇 하드웨어까지 4계층 파이프라인

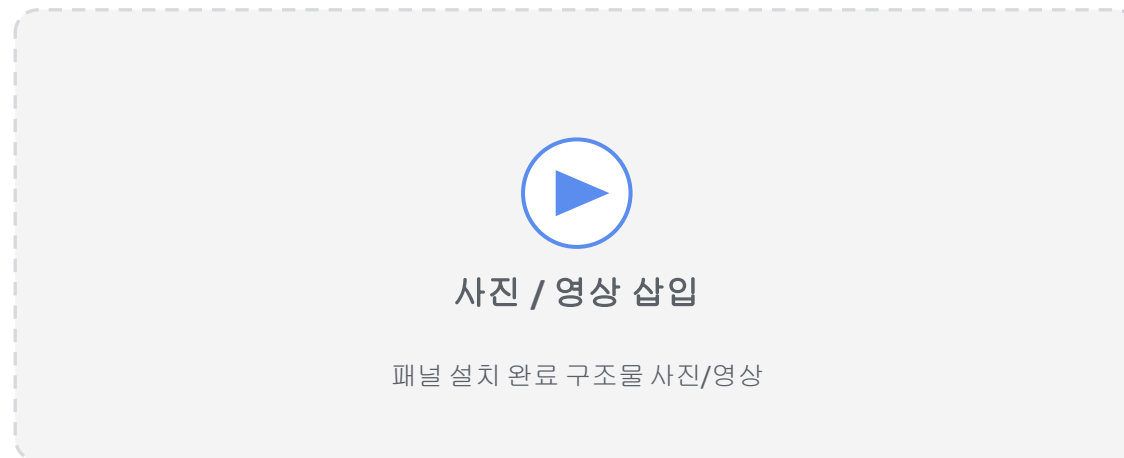
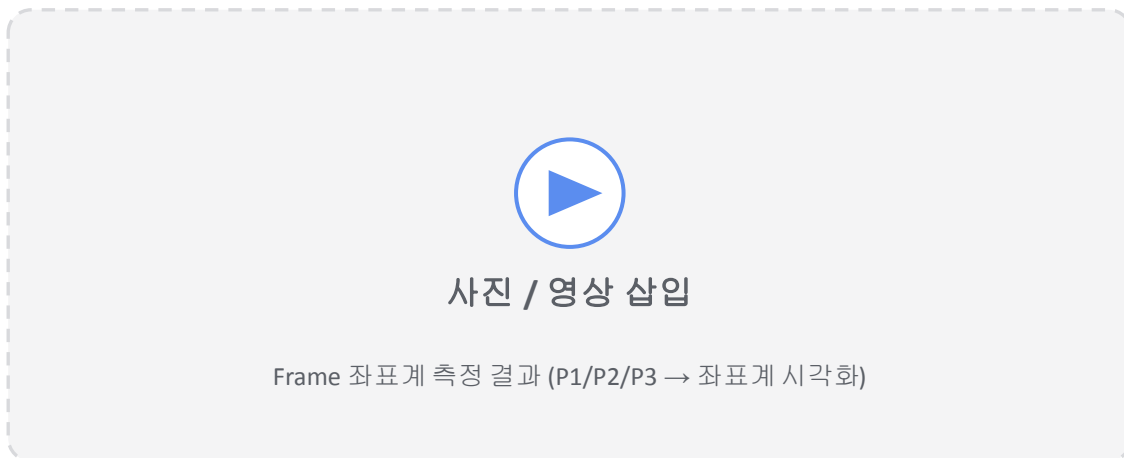
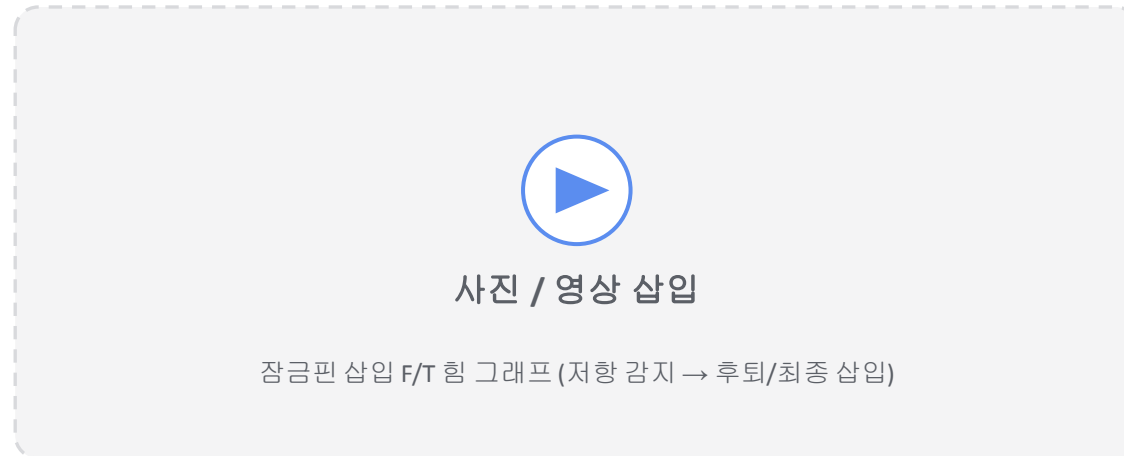
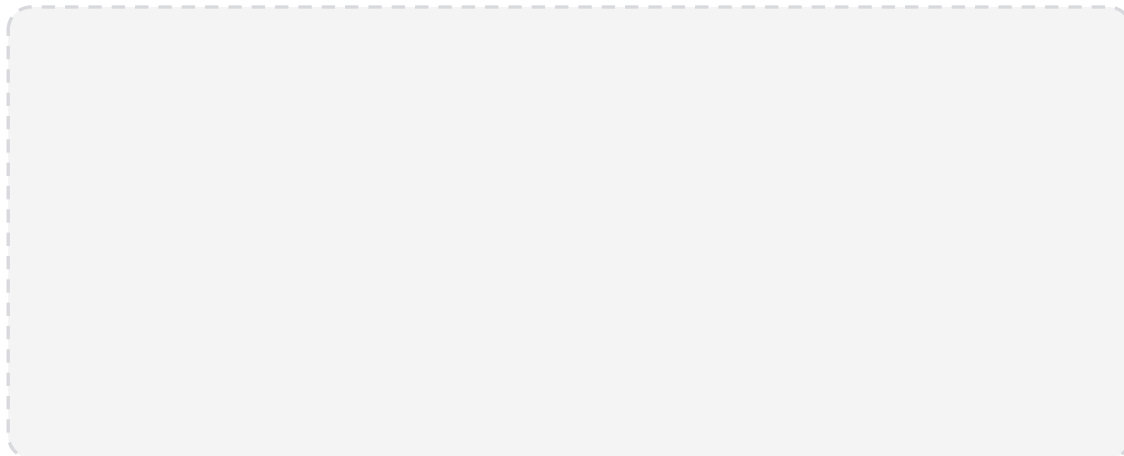


레이어별 구현 결과

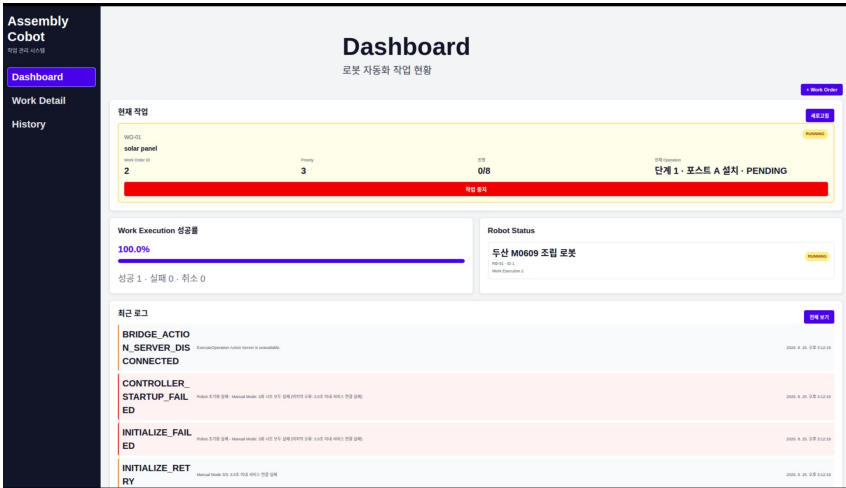
- **solar_panel_interface** ExecuteOperation.action · Parameter.msg · SystemEvent.msg · RecoverRobot.srv · StopOperation.srv — 통신 규격 정의
- **solar_panel_robot** robot_motion.py · force_control.py · solar_motion.py · controller_node.py
- **ros2_bridge** Flask(8001) + ActionClient 단일 프로세스, Operation 큐잉 · 순차 전송, Feedback/Result 콜백
- **web_app** Work Order/Recipe/실행 이력 관리 + PostgreSQL 9테이블 + 대시보드(5000)

결과 제시

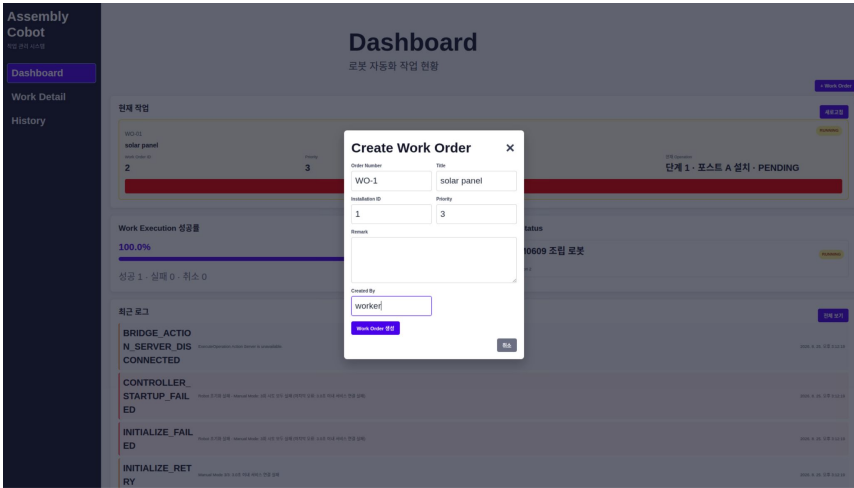
데모 캡처 및 측정 결과를 아래 영역에 삽입



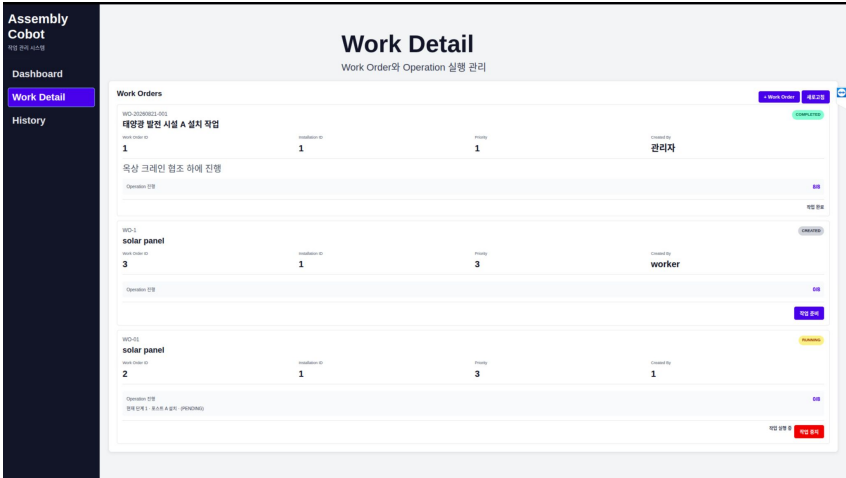
웹 서비스



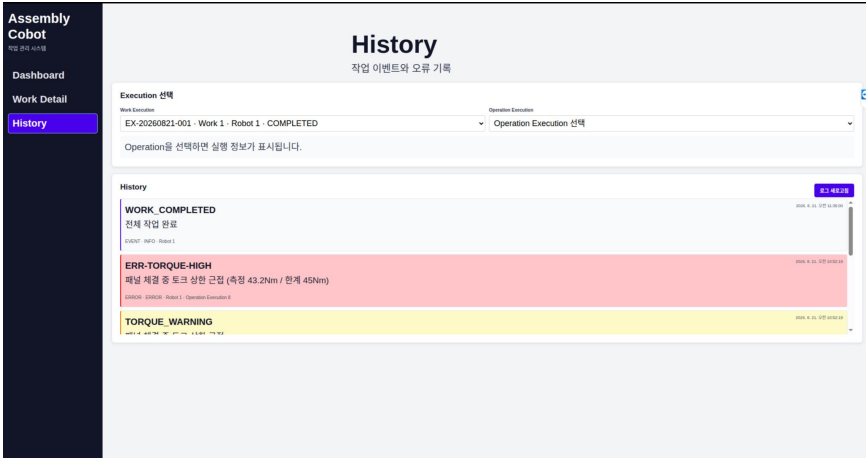
DashBoard



Work Order 생성



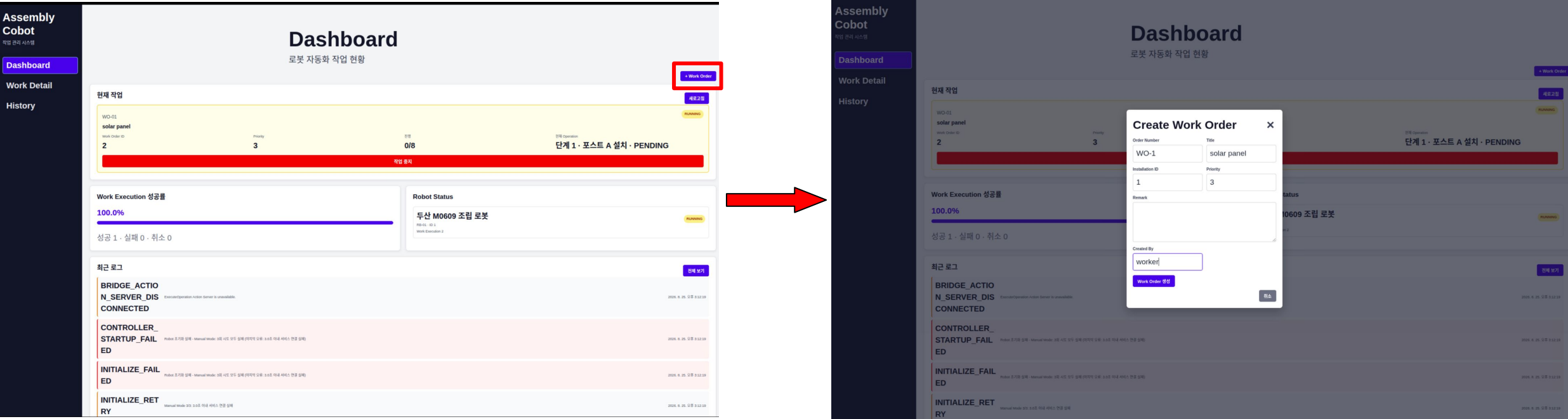
Work Detail



History

04 수행 경과

웹 서비스



웹 서비스

Assembly
Cobot

작업 관리 시스템

Dashboard

Work Detail

History

Work Detail

Work Order와 Operation 실행 관리

Work Orders

WO-20260821-001

태양광 발전 시설 A 설치 작업

Work Order ID

1

Installation ID

1

Priority

1

Created By

관리자

COMPLETED

작업 완료

WO-1

solar panel

Work Order ID

3

Installation ID

1

Priority

3

Created By

worker

CREATED

작업 준비

WO-01

solar panel

Work Order ID

2

Installation ID

1

Priority

3

Created By

1

RUNNING

작업 실행 중

웹 서비스

Assembly
Cobot

작업 관리 시스템

Dashboard

Work Detail

History

Work Detail

Work Order와 Operation 실행 관리

Work Orders

WO-20260821-001

태양광 발전 시설 A 설치 작업

Work Order ID

1

Installation ID

1

Priority

1

Created By

관리자

COMPLETED

작업 완료

작업 진행

0/8

작업 완료

WO-1

solar panel

Work Order ID

3

Installation ID

1

Priority

3

Created By

worker

READY

작업 시작

작업 진행

0/8

작업 시작

WO-01

solar panel

Work Order ID

2

Installation ID

1

Priority

3

Created By

1

RUNNING

작업 중지

작업 진행

0/8

작업 중지

웹 서비스

Assembly
Cobot

작업 관리 시스템

Dashboard

Work Detail

History

History

작업 이벤트와 오류 기록

Execution 선택

Work Execution

EX-20260821-001 · Work 1 · Robot 1 · COMPLETED

Operation Execution

Operation Execution 선택

Operation을 선택하면 실행 정보가 표시됩니다.

History

WORK_COMPLETED

전체 작업 완료

EVENT · INFO · Robot 1

2026. 8. 21. 오전 11:35:00

ERR-TORQUE-HIGH

패널 체결 중 토크 상한 근접 (측정 43.2Nm / 한계 45Nm)

ERROR · ERROR · Robot 1 · Operation Execution 8

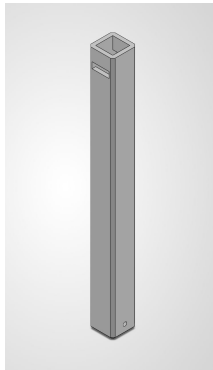
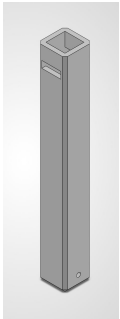
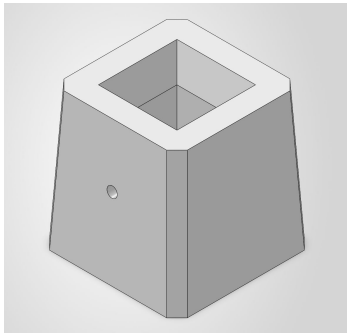
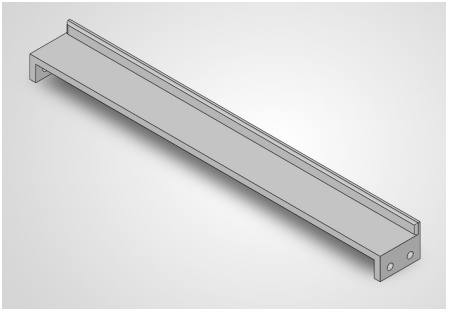
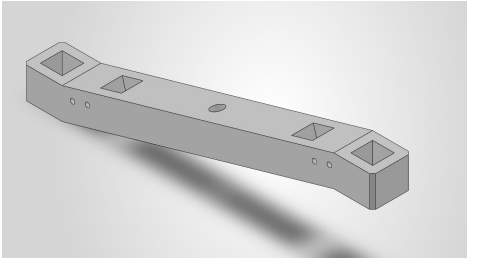
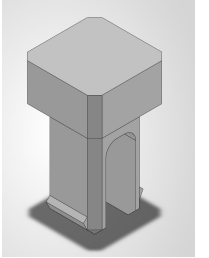
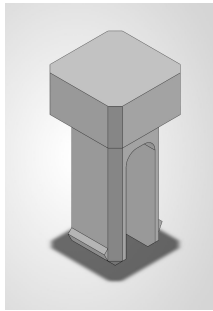
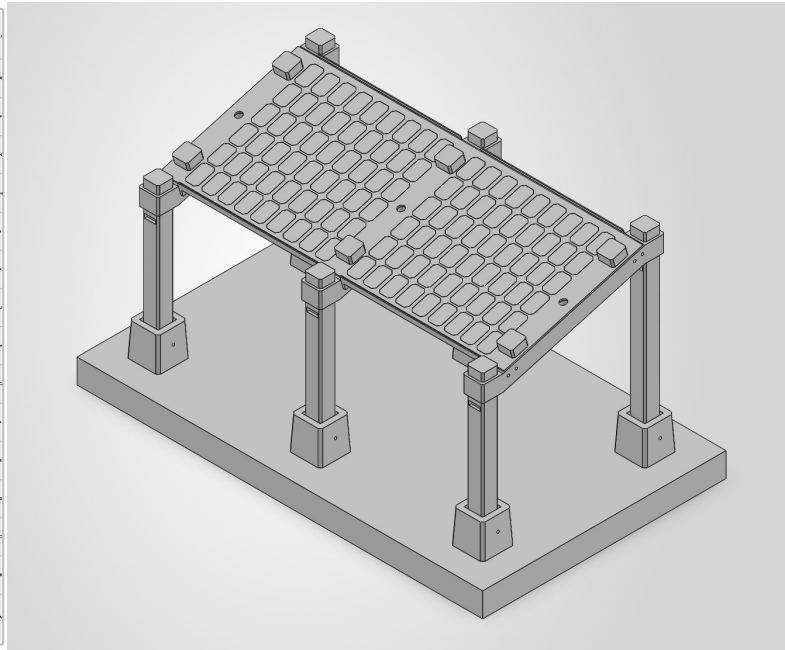
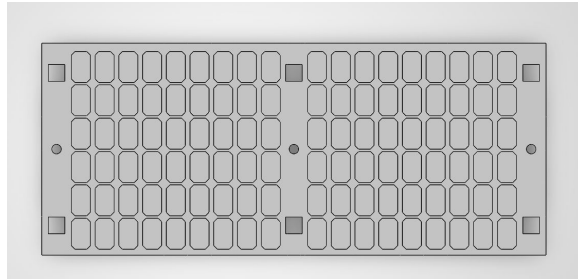
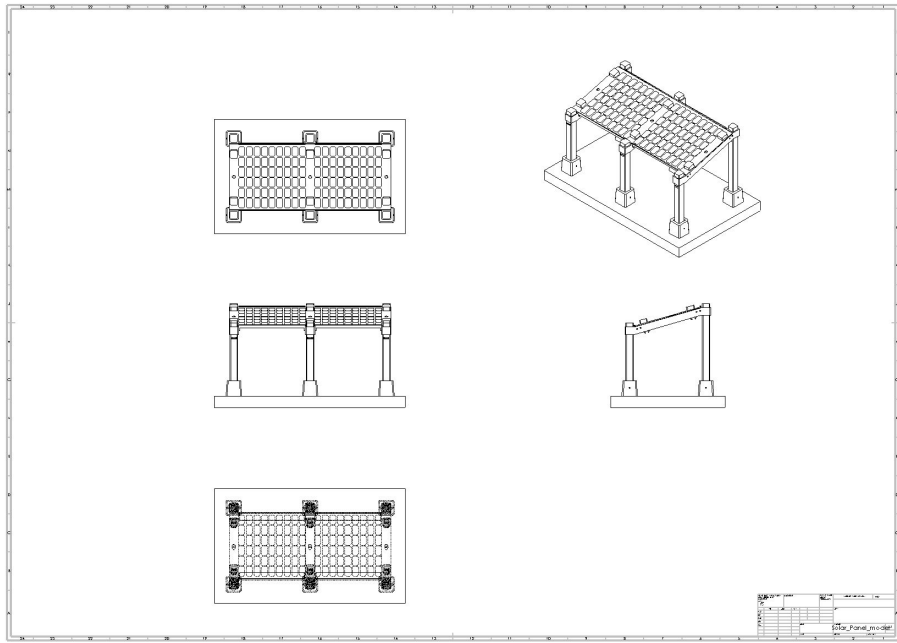
2026. 8. 21. 오전 10:52:19

TORQUE_WARNING

패널 체결 중 토크 상한 근접

2026. 8. 21. 오전 10:52:19

하드웨어



소프트웨어

ros2_bridge - Backend ↔ ROS2 Bridge 패키지

파일	역할
bridge_node.py	한 프로세스에 두 가지가 같이 떠 있음: ① Flask HTTP 서버(포트 8001) — Backend의 요청을 받는 /health , /status , /jobs (Operation 제출), /works/<id>/stop (강제 정지), /robots/<id>/recover (복구) 엔드포인트 제공. ② rclpy ActionClient — 받은 Operation들을 큐에 쌓고 순서대로 Controller에 Action Goal 전송, Feedback/Result를 받으면 Backend의 /api/v1/executions/action-feedback , /action-result 로 콜백. 또한 Controller가 publish하는 /system_event 를 subscribe해서 Backend /api/v1/logs 로 전달. 한 Operation이 실패하면 같은 작업의 나머지 대기 Operation은 자동으로 건너뛴

소프트웨어

solar_panel_robot - 로봇 제어 패키지

파일	역할
controller_node.py	ExecuteOperation Action Server (entry point: controller) . Bridge로부터 Goal을 받아 resolve_operation_code() 로 DB operation.code(postA ~ postF 등)를 검증하고, robot_id 일치·Controller 상태(ERROR 아님)·동시 실행 중복 여부를 확인해 Goal을 승인/거부. 승인 후 SolarMotion 에 실제 동작을 위임하고 Feedback(IF-15)/Result 응답. /system_event 로 초기화·Operation 진행·오류·복구 이벤트를 publish. Cancel은 아직 미지원(항상 REJECT)
action_server.py	실제 로봇 없이 통신 흐름만 검증하는 Mock Action Server (entry point: action_server) . Goal→Feedback→Result 진행을 시뮬레이션, 환경변수로 강제 실패 테스트, 그리고 stop_operation_callback / recover_robot_callback 로 Backend의 강제정지·복구 흐름도 모킹
robot_controller.py	로봇 초기화 전용. Manual/Auto 모드 전환, Tool/TCP(그리퍼 장착 정보) 설정, SDK 호출 재시도(_call_with_retry)
robot_motion.py	저수준 모션 라이브러리 . movej/movel 등 SDK 함수를 감싸서 pick/place, 원점 복귀, 힘 감지 이동 등 범용 로봇 동작 제공. 그리퍼 on/off도 여기 있음
solar_motion.py	공정별 상위 시나리오 . "프레임 Pick → 배치 → 설치", "핀 Pick → 삽입(1차/최종)" 등 태양광 조립 공정 순서를 robot_motion.py / force_control.py 를 조합해 구현. controller_node.py 가 실제로 호출하는 대상. coordinate_manager.py 는 참조하지 않음 — Frame 좌표계 측정은 폐기된 설계라 연동 계획 없음
force_control.py	F/T 센서 기반 힘 제어 . Compliance(순응 제어) on/off, 목표 힘 설정, 힘 조건 체크 — CLAUDE.md의 "잠금핀 삽입" 로직 담당
coordinate_manager.py	(폐기 예정) Frame 좌표계 생성 유틸 . DB에서 받은 UCS(P1/P2/P3, 각 6DOF pose)로 Doosan set_user_cart_coord() 를 호출해 사용자 좌표계를 생성하고 좌표계 ID를 캐싱(동일 P1/P2/P3면 재생성 안 함). get_current_pose() 로 해당 좌표계 기준 현재 위치 조회 가능. 2026-08 설계 변경으로 Frame 좌표계 측정 자체가 더 이상 사용하지 않는 기술로 확정되어, CoordinateManager 는 coord_manager_test.py 에서만 쓰이고 solar_motion.py / controller_node.py 실제 공정 흐름에 연결될 계획이 없음 — 삭제 후보
coord_manager_test.py	CoordinateManager 단독 동작 확인용 수동 실행 스크립트(main())
config_loader.py	config/poses.yaml (위치/자세, 속도, 그리퍼 포트값)을 로딩해주는 유틸
ros_bridge_node.py	Controller를 직접 호출하는 별도 Action Client. src/ros2_bridge 와 별개의 코드 — setup.py 의 console_scripts 에도 등록되어 있지 않아 실행조차 되지 않는 미사용 코드 (정리 필요 항목)

04 수행 경과

소프트웨어

solar_panel_interface - 인터페이스 정의

```
# Controller -> ROS Bridge
# Robot/System 이벤트 및 Operation 세부 단계 로그
```

```
uint8 SEVERITY_INFO=0
uint8 SEVERITY_WARNING=1
uint8 SEVERITY_ERROR=2
uint8 SEVERITY_CRITICAL=3
```

```
# 항상 존재하는 Robot 식별자
uint64 robot_id
```

```
# Operation과 무관한 시스템 이벤트에서는 0
uint64 work_execution_id
uint64 operation_execution_id
```

```
# Operation과 무관한 시스템 이벤트에서는 빈 문자열
string operation_code
string phase
string status
```

```
uint8 severity
string code
string message
```

```
# JSON 문자열. 추가 구조화 정보가 없으면 "{}"
string detail
```

SystemEvent.msg

```
# Recipe/Operation Parameter
string key
string value
```

Parameter.msg

```
# Request
uint64 work_execution_id
uint64 operation_execution_id
uint64 robot_id
---
```

```
# Response
bool accepted
string error_code
string message
```

StopOperation.srv

```
uint64 robot_id
---
bool recovered
string error_code
string message
```

RecoverRobot.srv

```
# Goal
uint64 work_order_id
uint64 work_execution_id
uint64 operation_id
```

```
# DB operation.code
# 예: postA, postB, postC ...
string operation_code
```

```
uint64 operation_execution_id
uint64 robot_id
```

```
solar_panel_interface/Parameter[] parameters
```

```
# operation.components(JSONB)를 JSON 문자열로 전달
string components
---
```

```
# IF-16: Controller -> ROS Bridge
# Result
```

```
bool success
string error_code
string message
---
```

```
# IF-15: Controller -> ROS Bridge
# Feedback
```

```
uint8 STATUS_PENDING=0
uint8 STATUS_RUNNING=1
uint8 STATUS_COMPLETED=2
uint8 STATUS_FAILED=3
uint8 STATUS_CANCELLED=4
```

```
uint64 operation_execution_id
uint8 status
string message
```

ExecuteOperation.action

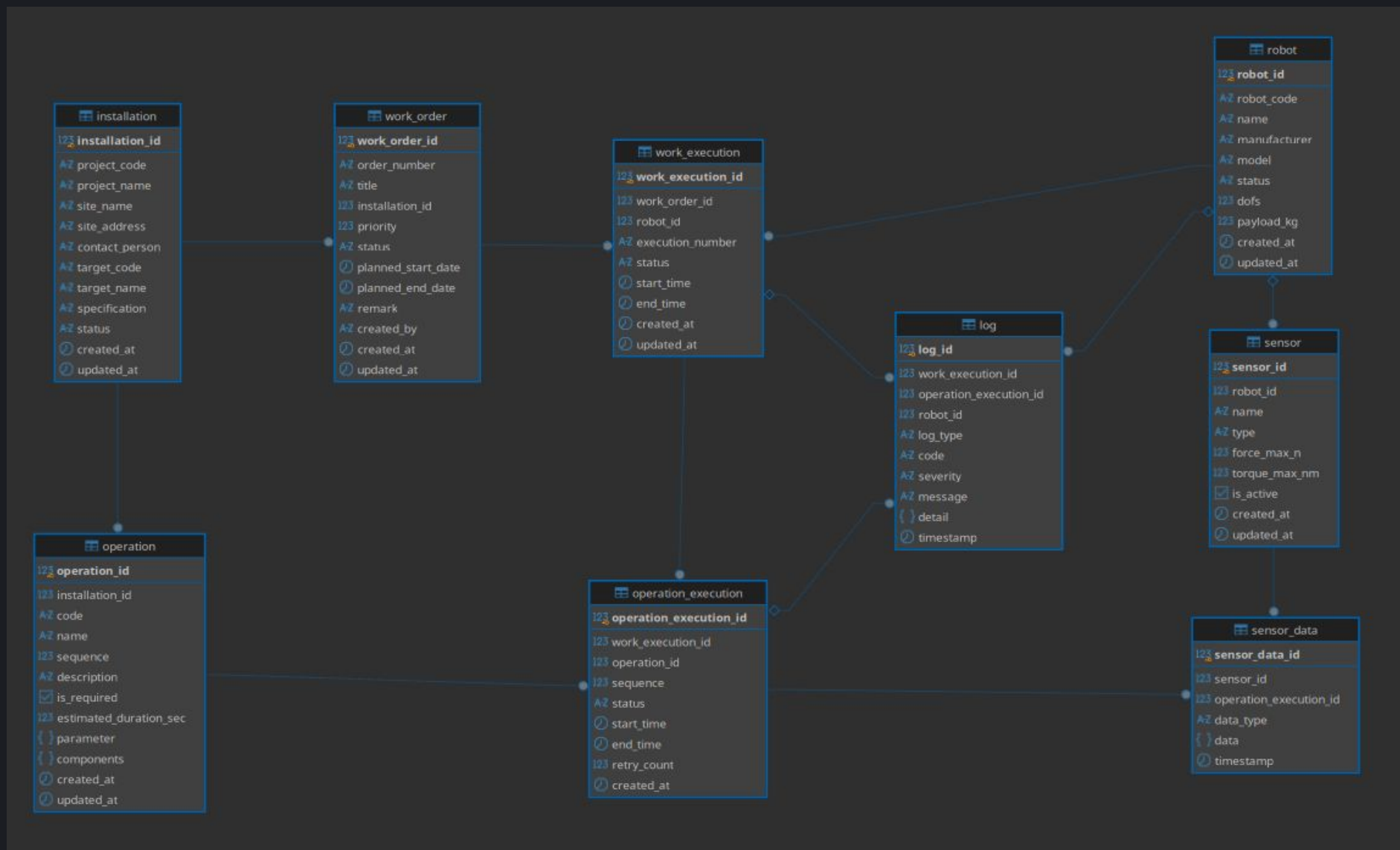
소프트웨어

web_app - Flask Backend (Work Order / Recipe 관리)

위치	역할
app/models/	DB 테이블 매핑: WorkOrder , Operation , WorkExecution , OperationExecution , Robot , Installation , SystemLog , SensorData (schema.sql 기준 9개 테이블 중 sensor 는 아직 모델 매핑 없음)
app/routes/work_orders.py	Work Order CRUD, /execute (Bridge에 전체 Operation 제출), /progress (진행률 조회), /stop (RUNNING 중인 Work Order 강제 정지 — Bridge /works/<id>/stop 호출 후 실행 상태를 CANCELLED로 마킹)
app/routes/executions.py	Bridge가 호출하는 콜백 2종(/action-feedback , /action-result) — 실행 상태를 DB에 반영, 다음 Operation으로 진행 처리
app/routes/robots.py	GET / (로봇 목록 + 현재 실행 중인 Work 정보 조인), POST /<id>/recover (ERROR 상태 로봇을 Bridge /robots/<id>/recover 호출로 복구, 시스템 로그 기록)
app/routes/dashboard.py	대시보드용 집계 데이터(/api/v1/dashboard) 단일 엔드포인트
app/routes/bridge.py	Bridge 헬스체크 프록시
app/routes/logs.py	시스템 로그 조회/기록 API — ROS SystemEvent 가 Bridge를 거쳐 여기로 들어옴(연결됨)
app/routes/sensor_data.py	Operation Execution 기준 센서 데이터 조회(GET /api/v1/sensor-data)
app/routes/health.py , pages.py	Backend 자체 헬스체크, 정적 대시보드 페이지 서빙
app/services/bridge_service.py	Bridge HTTP 클라이언트: /jobs 제출용 Work Command 조립(build_work_command), submit_bridge_job/work , stop_bridge_work , recover_bridge_robot , 연결/응답 오류 예외(BridgeConnectionError , BridgeResponseError)
app/services/dashboard_service.py	로봇별 활성 실행 요약, 성공률 등 대시보드 집계 쿼리
app/services/robot_service.py	로봇 목록 조회, 로봇별 활성 Work Execution 조회
app/services/execution_service.py	Execution/Operation Execution 상태 전이 로직 전반(진행/완료/실패/취소/강제정지/로봇 복구 포함)
app/services/work_order_service.py , operation_service.py , log_service.py	각각 Work Order CRUD, Operation 조회, 시스템 로그 생성/조회
database/schema.sql , seed.sql	PostgreSQL 스키마(9개 테이블: installation, operation, robot, sensor, work_order, work_execution, operation_execution, log, sensor_data)와 데모용 초기 데이터
frontend/	정적 HTML/CSS/JS 대시보드 (Work Order 생성·조회·실행·강제정지, 로봇 상태·복구, 시스템 로그 조회 UI)

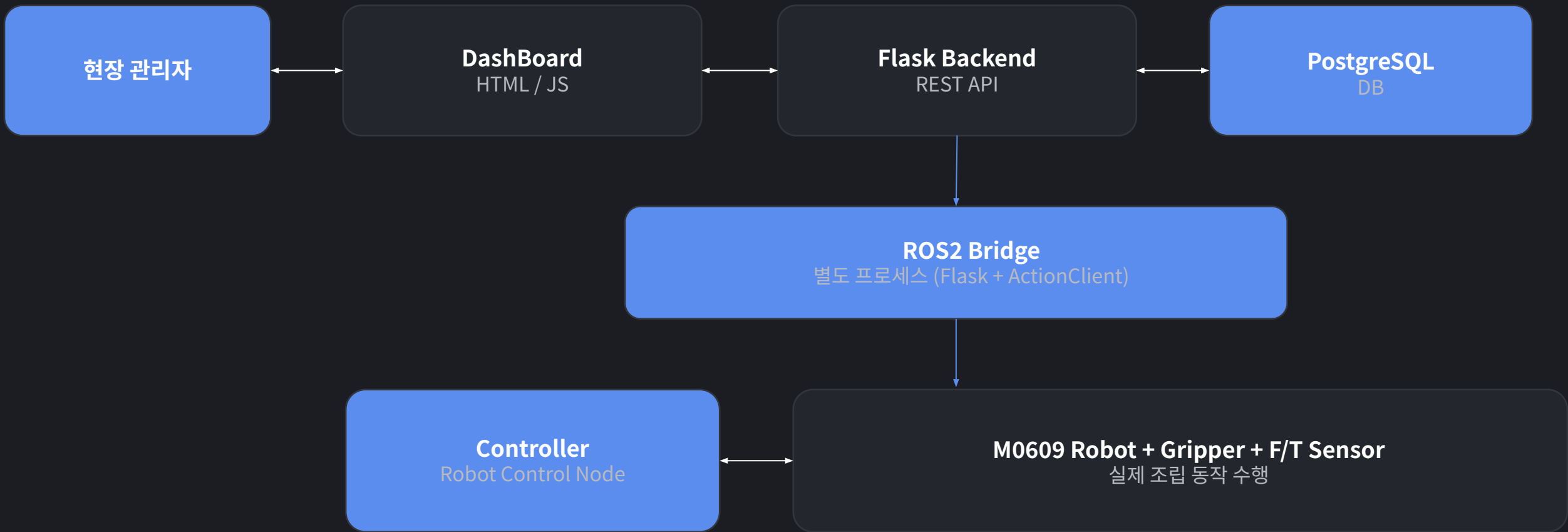
소프트웨어

DB



시스템 아키텍처

현장 관리자부터 로봇 조작까지의 시스템 흐름 파이프라인



완성도 평가

[0]

/ 10

자체 평가 점수

실제 점수 및 근거로 교체

목표 대비 구현 수준

사전 기획 단계에서 세운 목표(기둥 → 프레임 → 잠금핀 체결 → 좌표계 측정 → 패널 설치) 중 web_app 전 구간은 완성 단계에 도달했으며, 실물 로봇 연동(Step 12)은 진행 중입니다.

핵심 차별화 포인트인 F/T 센서 기반 힘 제어 삽입과 실측 기반 좌표계 생성은 [완성 / 부분 구현] 상태로, 발표 시 실제 진행 상황에 맞게 교체가 필요합니다.

잘한 점 & 개선점

잘한 점

- BR→SYS-FR→FR→TC 요구사항 추적 체계를 먼저 세워 팀 전체가 같은 기준으로 판단
- 통신 인터페이스(solar_panel_interface)를 계약처럼 먼저 정의 → 로봇 제어/Bridge/Backend 병렬 개발
- 단순 좌표 이동이 아닌 힘 제어 기술 구현

아쉬운 점 / 개선점

- 하드웨어 설계에서 너무 큰 공차를 준 것과 부품 보관 지그 미설계로 인해 조립 과정에서 애로 사항 발생
- SystemEvent처럼 정의만 하고 연결을 미룬 부분 존재
- F/T 센서 임계값 튜닝은 수월했으나, 하드웨어 캘리브레이션 일정은 더 넉넉히 필요

경험 및 성과

ROS2 통신(Action, Service, Topic) 기반 다단계 파이프라인(작업 관리 노드 ↔ Controller ↔ 로봇)과 DB 연동 Work Order/Recipe 관리 구조를 처음부터 끝까지 설계·구현

[팀 / 개인 차원에서 느낀 점, 진로 연관성 등 자유 기술]

감사합니다